

A **class** in Java is a **blueprint** or **template** from which **objects** are created.

Each object has:

Data (attributes) — what it *has*

Methods (functions) — what it *does*

- **Car** → Class (blueprint)
- **c1** → Object
- **color** → Attribute
- **start()** → Method

```
class Car {  
    String color;  
    void start() {  
        System.out.println("Car is starting...");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Car c1 = new Car(); // object created  
        c1.color = "Red";  
        c1.start();  
    }  
}
```

Why do we use Classes?

1.Reusability – You can reuse the same class to create multiple objects.

Example: You can create many cars (Car car1 = new Car(); Car car2 = new Car();).

2.Organization – Classes help organize your code better by grouping related variables and methods together.

3.Encapsulation – You can hide internal details and expose only necessary information (data protection).

4.Object-Oriented Programming (OOP) – Java is based on OOP principles (class, object, inheritance, polymorphism, etc.), and **class** is the foundation.

The General Form of a Class

```
class classname {
```

```
type instance-variable1;
```

```
type instance-variable2;
```

```
// ...
```

```
type instance-variableN;
```

```
type methodname1(parameter-list) {
```

```
// body of method
```

```
}
```

```
// ...
```

```
type methodnameN(parameter-list) {
```

```
// body of method
```

```
}
```

```
}
```

- Class defines a new data type. Once defined, this new type can be used to create objects of that type.
- A class is declared by use of the **class** keyword
- The data, or variables, defined within a **class** are called *instance variables*.
- The code is contained within *methods*.
- Collectively, the methods and variables defined within a class are called *members* of the class.
- Variables defined within a class are called instance variables

Class vs Object

Example — “House Blueprint”

A **class** is like an **architect’s blueprint** for a house.

It describes what the house *will have* — walls, doors, windows, rooms, etc.

An **object** is an **actual house** built from that blueprint.

Each house (object) looks similar (same structure) but can have **different colors, materials, or furniture** (different data values).

```
class House {  
    String color;  
    int rooms;  
  
    void display() {  
        System.out.println("Color: " + color + ", Rooms: " + rooms);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        House h1 = new House(); // Object 1  
        h1.color = "Blue";  
        h1.rooms = 3;  
  
        House h2 = new House(); // Object 2  
        h2.color = "Red";  
        h2.rooms = 4;  
  
        h1.display();  
        h2.display();  
    }  
}
```

Concept

Class

Blueprint or design plan

Object

Actual item built from the design

Attributes (variables)

Characteristics (e.g., color, size)

Methods (functions)

Actions the object can perform

A Simple Class

```
class Box {  
    double width;  
    double height;  
    double depth;  
}  
  
class BoxDemo {  
    public static void main(String args[]) {  
        Box mybox = new Box();  
        double vol;  
        // assign values to mybox's instance variables  
        mybox.width = 10;  
        mybox.height = 20;  
        mybox.depth = 15;  
        // compute volume of box  
        vol = mybox.width * mybox.height * mybox.depth;  
        System.out.println("Volume is " + vol);  
    }  
}
```

```
class Box {
double width;
double height;
double depth;
}
class BoxDemo2 {
public static void main(String args[]) {
Box mybox1 = new Box();
Box mybox2 = new Box();
double vol;
// assign values to mybox1's instance variables
mybox1.width = 10;
mybox1.height = 20;
mybox1.depth = 15;
/* assign different values to mybox2's instance variables */
mybox2.width = 3;
mybox2.height = 6;
mybox2.depth = 9;
```

```
// compute volume of first box
```

```
vol = mybox1.width * mybox1.height * mybox1.depth;
```

```
System.out.println("Volume is " + vol);
```

```
// compute volume of second box
```

```
vol = mybox2.width * mybox2.height * mybox2.depth;
```

```
System.out.println("Volume is " + vol);
```

```
}
```

```
}
```

Declaring Objects

- When you create a class, you are creating a new data type.
- However, obtaining objects of a class is a two-step process.
- First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can *refer* to an object.
- Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the **new** operator.
- The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it.

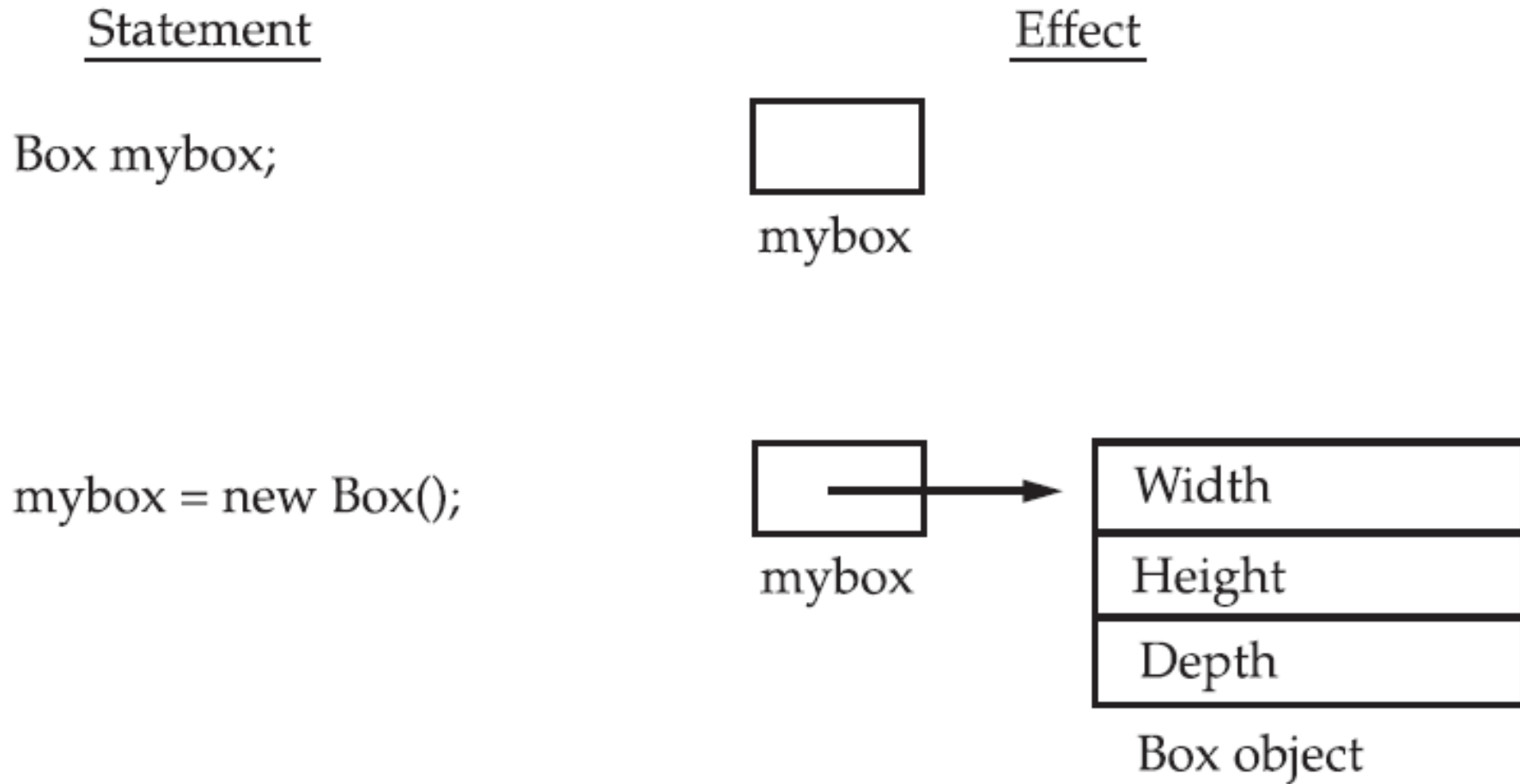


Figure 6-1 Declaring an object of type **Box**

Box mybox = new Box();

This statement combines the two steps just described.

Box mybox; // declare reference to object

mybox = new Box(); // allocate a Box object

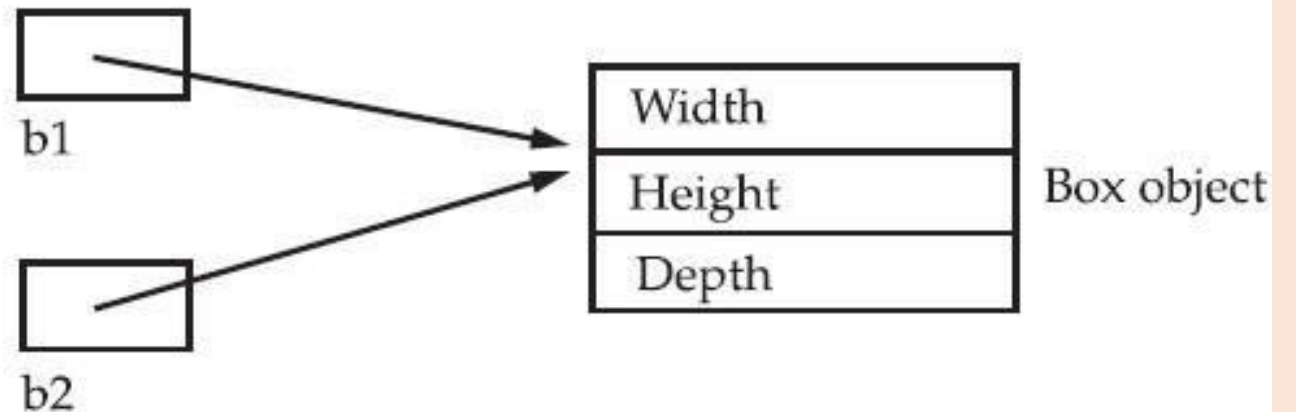
Assigning Object Reference Variables

Object reference variables act differently when an assignment takes place.

```
Box b1 = new Box();
```

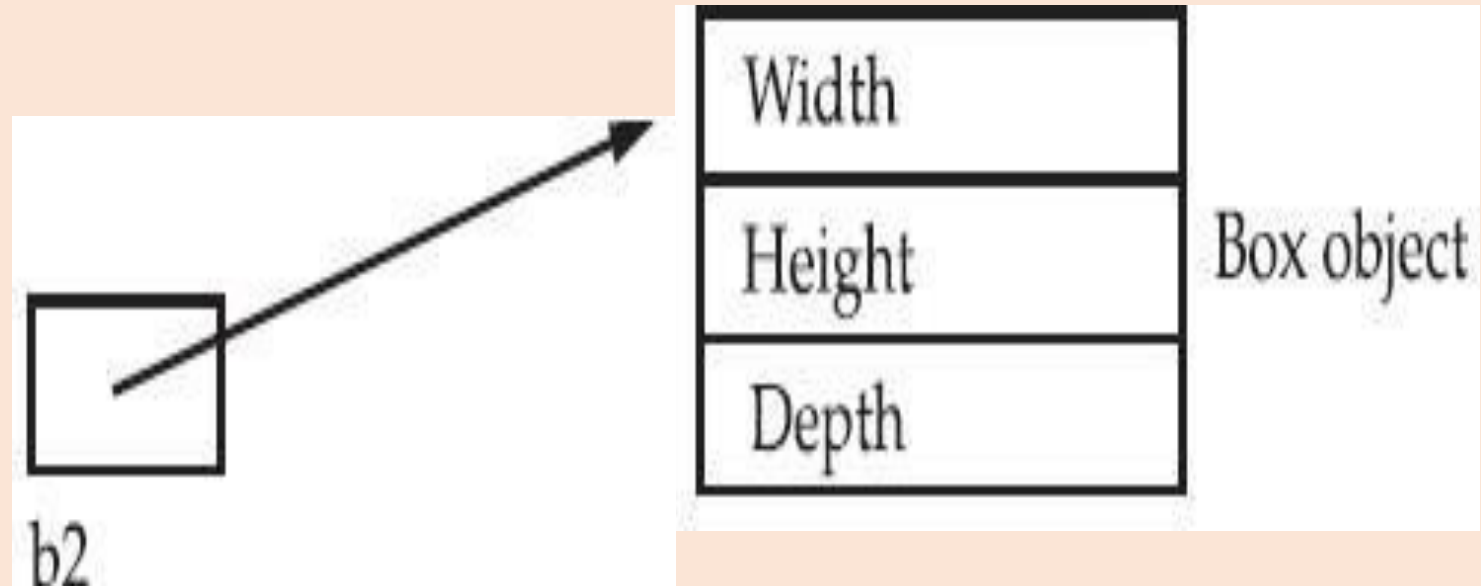
```
Box b2 = b1;
```

This situation is depicted here:



```
Box b1 = new Box();  
Box b2 = b1;  
// ...  
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object



Introducing methods

This is the general form of a method:

```
type name(parameter-list) {  
    // body of method  
}
```

Adding a Method to the Box Class

```
class Box {  
  
    double    width;  
  
    double    height;  
  
    double depth;  
  
    // display volume of a  
    box void volume() {  
  
        System.out.print("Volume is ");  
  
        System.out.println(width    *  
            height * depth);  
    }  
}
```

```
class BoxDemo3 {  
    public static void main(String args[]) {  
  
        Box mybox1 = new Box();  
  
        // assign values to mybox1's instance variables  
  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;  
  
        // display volume of first box  
  
        mybox1.volume();  
    }  
}
```

Returning a Value

- While the implementation of **volume()** does move the computation of a box's volume inside the **Box** class where it belongs, it is not the best way to do it.

```
class Box {  
    double width;  
  
    double height;  
  
    double depth;  
  
    // compute and return volume  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

```
class BoxDemo4 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        double vol;  
        // assign values to mybox1's instance variables  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;  
        // get volume of first box vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

Adding a Method That Takes Parameters

- While some methods don't need parameters, most do. Parameters allow a method to be generalized.
- That is, a parameterized method can operate on a variety of data and/or be used in a number of slightly different situations

```
int square(int i)
{
    return i * i;
}
```

```
int x, y;
x = square(5); // x equals 25
x = square(9); // x equals 81
y = 2;
x = square(y); // x equals 4
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    // sets dimensions of box  
    void setDim(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
}
```

```
class BoxDemo5 {  
  
    public static void main(String args[]) {  
  
        Box mybox1 = new Box();  
  
        Box mybox2 = new Box();  
  
        double vol;  
  
        // initialize each box  
  
        mybox1.setDim(10, 20, 15);  
        mybox2.setDim(3, 6, 9);  
  
        // get volume of first box  
  
        vol = mybox1.volume();  
  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
  
    }  
}
```

Constructors

- It can be tedious to initialize all of the variables in a class each time an instance is created.
- Because the requirement for initialization is so common, Java allows objects to initialize themselves when they are created.
- This automatic initialization is performed through the use of a constructor.
- A *constructor* initializes an object immediately upon creation.
- It has the same name as the class in which it resides and is syntactically similar to a method.
- Once defined, the constructor is automatically called immediately after the object is created, before the **new** operator completes.

Constructors look a little strange because they have no return type, not even **void**

```
class Box {  
    double width;  
  
    double height;  
    double depth;  
    Box() {  
        System.out.println("Constructing Box");  
  
        width = 10;  
        height = 10;  
        depth = 10;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

```
class BoxDemo6 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box objects  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

Parameterized Constructors

```
class Box {  
    double width; double height; double depth;  
  
    Box(double w, double h, double d) {  
        width = w; height = h; depth = d;  
    }  
  
    double volume() {  
        return width * height * depth;  
    }  
}
```

```
class BoxDemo7 {  
    public static void main(String args[]) {  
        // declare, allocate, and initialize Box objects  
  
        Box mybox1 = new Box(10, 20, 15);  
  
        Box mybox2 = new Box(3, 6, 9);  
  
        double vol;  
  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

The **this** keyword

- Sometimes a method will need to refer to the object that invoked it.
- To allow this, Java defines the **this** keyword. **this** can be used inside any method to refer to the *current* object

```
Box(double w, double h, double d) {  
    this.width = w; this.height = h; this.depth = d;  
}
```

Uses of this:

- To overcome shadowing or instance variable hiding.
- To call an overload constructor

- can have local variables, including formal parameters to methods, which overlap with the names of the class' instance variables.
- However, when a local variable has the same name as an instance variable, the local variable *hides* the instance variable.

```
Box(double width, double height, double depth) {  
    this.width = width; this.height = height;  
    this.depth = depth;  
}
```

Garbage Collection

- Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation.
- Java takes a different approach; it handles deallocation for you automatically.
- The technique that accomplishes this is called *garbage collection*.
- It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.
- There is no explicit need to destroy objects as in C++.

The finalize() Method

- Sometimes an object will need to perform some action when it is destroyed. For example, if an object is holding some non-Java resource such as a file handle or character font, then you might want to make sure these resources are freed before an object is destroyed.
- To handle such situations, Java provides a mechanism called *finalization*.
- By using finalization, you can define specific actions that will occur when an object is just about to be reclaimed by the garbage collector.

The **finalize()** method has this general form:

```
protected void finalize( )  
{  
    // finalization code here  
}
```

- the keyword **protected** is a specifier that prevents access to **finalize()** by code defined outside its class.
- It is important to understand that **finalize()** is only called just prior to garbage collection.

A Stack Class

```
class Stack {  
    int stck[] = new int[10];  
    int tos;  
    // Initialize top-of-stack  
    Stack() {  
        top = -1;  
    }  
    // Push an item onto the stack  
    void push(int item) {  
        if(top==9)  
            System.out.println("Stack is full.");  
        else  
            stck[++top] = item;  
    }  
    // Pop an item from the stack
```

```
int pop() {  
    if(top < 0) {  
        System.out.println("Stack underflow.");  
        return 0;  
    }  
    else  
return stck[top--];  
}  
}
```

```
class TestStack {  
    public static void main(String args[]) {  
        Stack mystack1 = new Stack();  
        Stack mystack2 = new Stack();  
    }  
}
```

// push some numbers onto the stack

```
for(int i=0; i<10; i++)
```

```
mystack1.push(i);
```

```
for(int i=10; i<20; i++)
```

```
mystack2.push(i);
```

// pop those numbers off the stack

```
System.out.println("Stack in mystack1:");
```

```
for(int i=0; i<10; i++)
```

```
    System.out.println(mystack1.pop());
```

```
System.out.println("Stack in mystack2:");
```

```
for(int i=0; i<10; i++)
```

```
    System.out.println(mystack2.pop());
```

```
}}
```

Overloading Methods

- In Java it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different.
- When this is the case, the methods are said to be *overloaded*, and the process is referred to as *method overloading*.
- Method overloading is one of the ways that Java supports polymorphism.
- Thus, overloaded methods must differ in the type and/or number of their parameters.
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.

```
class OverloadDemo {  
    void test() {  
        System.out.println("No parameters");  
    }  
    void test(int a) {  
        System.out.println("a: " + a);  
    }  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " " + b);  
    }  
    double test(double a) {  
        System.out.println("double a: " + a);  
        return a*a;  
    }  
}
```

```
class Overload {  
    public static void main(String args[]) {  
        OverloadDemo ob = new OverloadDemo();  
        double result;  
        // call all versions of test()  
        ob.test();  
        ob.test(10);  
        ob.test(10, 20);  
        result = ob.test(123.25);  
        System.out.println("Result of ob.test(123.25): " + result);  
    }  
}
```

```
class MathOperations {  
  
    // Method 1: add two integers  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2: add three integers  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method 3: add two double numbers  
    double add(double a, double b) {  
        return a + b;  
    }  
}
```

```
public class OverloadingExample {  
    public static void main(String[] args) {  
        MathOperations math = new MathOperations();  
  
        System.out.println("Sum of two integers: " + math.add(10, 20));  
        System.out.println("Sum of three integers: " + math.add(10, 20, 30));  
        System.out.println("Sum of two doubles: " + math.add(5.5, 6.7));  
    }  
}
```

- . When an overloaded method is called, Java looks for a match between the arguments used to call the method and the method's parameters.
- . However, this match need not always be exact. In some cases, Java's automatic type conversions can play a role in overload resolution.

For example, consider the following program:

```
// Automatic type conversions apply to overloading. class
```

```
OverloadDemo {  
    void test() {  
        System.out.println("No parameters");  
    }  
  
    void test(int a, int b) {  
        System.out.println("a and b: " + a + " " + b);  
    }  
  
    void test(double a) {  
        System.out.println("Inside test(double) a: " + a);  
    }  
}
```

```
class Overload {  
    public static void main(String args[]) {  
        OverloadDemo ob = new OverloadDemo();  
  
        int i = 88;  
        ob.test();  
        ob.test(10, 20);  
  
        ob.test(i); // this will invoke test(double)  
        ob.test(123.2); // this will invoke test(double)  
    }  
}
```

Overloading Constructors

- In addition to overloading normal methods, you can also overload constructor methods. In fact, for most real-world classes that you create, overloaded constructors will be the norm, not the exception.

```
class Box {  
    double width;  
  
    double height;  
  
    double depth;  
  
    // constructor used when all dimensions specified  
    Box(double w, double h, double d) {  
        width = w; height = h; depth = d;  
    }  
}
```

```
// constructor used when no dimensions specified
```

```
Box() {
```

```
    width = -1; // use -1 to indicate
```

```
    height = -1; // an uninitialized
```

```
    depth = -1; // box
```

```
}
```

```
// constructor used when cube is created
```

```
Box(double len) {
```

```
    width = height = depth = len;
```

```
}
```

```
// compute and return volume
```

```
double volume() {
```

```
    return width * height * depth;
```

```
}
```

```
}
```

```
class OverloadCons {  
    public static void main(String args[]) {  
        // create boxes using the various constructors  
  
        Box mybox1 = new Box(10, 20, 15);  
  
        Box mybox2 = new Box();  
  
        Box mycube = new Box(7);  
  
        double vol;  
  
        vol = mybox1.volume();  
  
        System.out.println("Volume of mybox1 is " + vol);  
  
        // get volume of second box  
  
        vol = mybox2.volume();  
  
        System.out.println("Volume of mybox2 is " + vol);  
    }  
}
```

```
// get volume of cube
```

```
vol = mycube.volume();
```

```
System.out.println("Volume of mycube is " + vol);
```

```
}
```

```
}
```

Using Objects as Parameters

- So far, we have only been using simple types as parameters to methods. However, it is both correct and common to pass objects to methods.
- For example, consider the following short program:

```
// Objects may be passed to methods. class Test {  
    int a, b;  
    Test(int i, int j) {  
        a = i;  
        b = j;  
    }  
  
    // return true if o is equal to the invoking object  
  
    boolean equals(Test o) {  
        if(o.a == a && o.b == b)  
            return true;  
        else  
            return false;  
    }  
}
```

```
class PassOb {  
    public static void main(String args[]) {  
        Test ob1 = new Test(100, 22);  
        Test ob2 = new Test(100, 22);  
        Test ob3 = new Test(-1, -1);  
        System.out.println("ob1 == ob2: " + ob1.equals(ob2));  
        System.out.println("ob1 == ob3: " + ob1.equals(ob3));  
    }  
}
```

This program generates the following output:

```
ob1 == ob2: true  
ob1 == ob3: false
```